# Personal Expense Tracker

*(DBMS Mini Project Report)*

*Submitted by:*

**Ashish Patra**
**Keshav Garg**
**Soumya Singh**

**Semester : 4th Sem B.Tech**

**Course : Database Management System**

**Submitted To : Dr. Vasudha Bahl**

**Date of Submission :**

**Department of Information Technology**

Maharaja Agrasen Institute of Technology

**CONTENTS**

1. **INTRODUCTION**
2. **AIM**
3. **OBJECTIVES**
4. **SYSTEM OVERVIEW**
5. **ENTITY-RELATIONSHIP DIAGRAM**
6. **DATABASE SCHEMA**
7. **IMPLEMENTATION**
8. **SQL QUERIES**
9. **OUTPUT SCREENSHOTS**
10. **ADVANTAGES**
11. **FUTURE SCOPE**
12. **CONCLUSION**

# 1. INTRODUCTION

The Personal Expense Tracker is a database system designed to manage and analyze personal financial data. It helps users keep track of their income and expenses in an organized manner. The system allows users to categorize their expenses and perform various operations such as calculating total expenses, income, and remaining balance.

This project demonstrates the use of Database Management System (DBMS) concepts such as table creation, relationships, and SQL queries

# 2. AIM

The aim of this project is to design and implement a database system for managing personal income and expenses efficiently using SQL and DBMS concepts.

# 3. OBJECTIVES

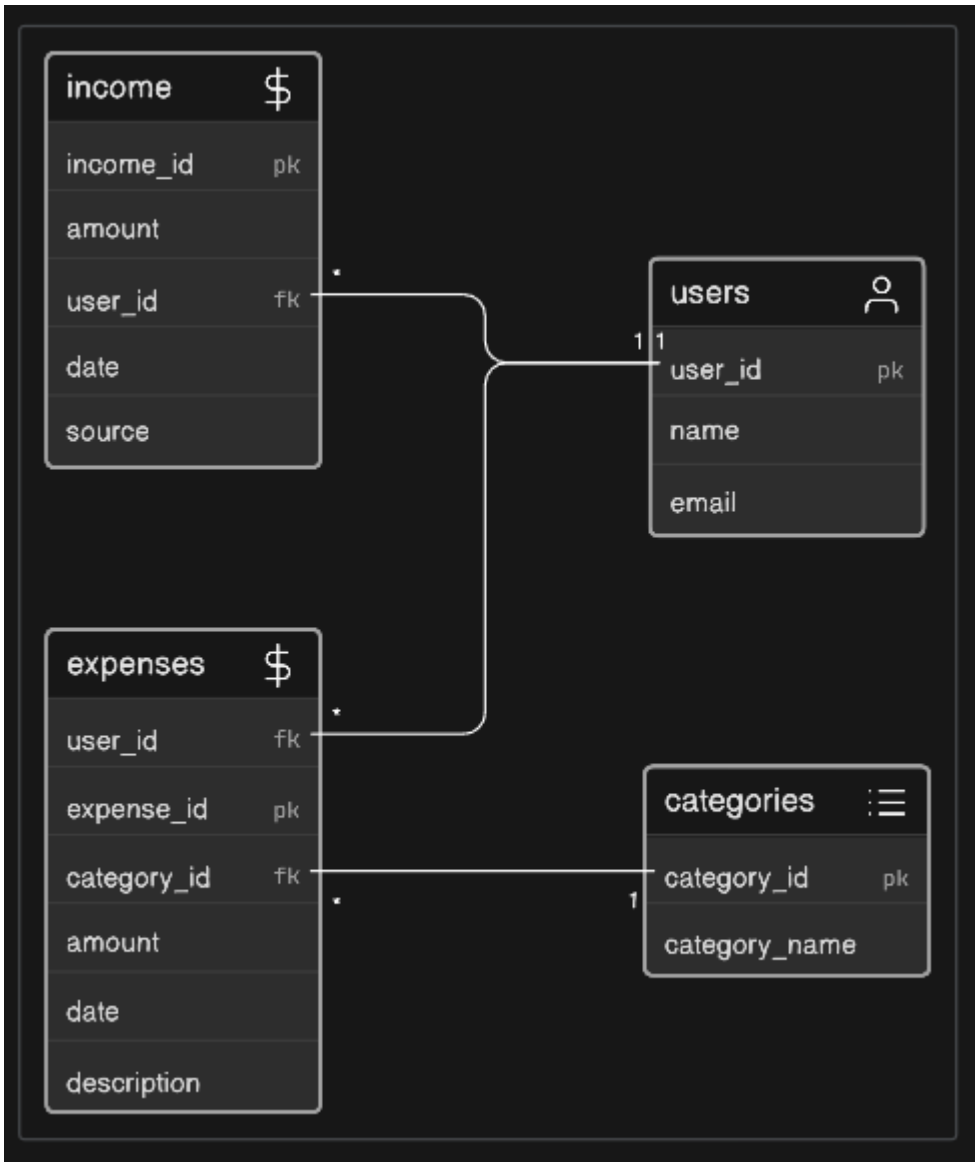
- To create a structured database for storing financial data
- To record income and expense details
- To categorize expenses for better analysis
- To perform queries for financial insights
- To calculate total income and expenses

# 4. SYSTEM OVERVIEW

The system consists of four main entities: Users, Categories, Income, and Expenses. Each user can have multiple income and expense records. Expenses are categorized into different categories such as food, travel, and bills.

The system provides an organized way to store financial data and retrieve meaningful information using SQL queries.

# 5. ENTITY-RELATIONSHIP DIAGRAM




**Relational Schema**

The ER diagram represents the relationships between users, income, expenses, and categories. A user can have multiple income and expense records, and each expense belongs to a specific category. These relationships are maintained using foreign keys.

# 6. DATABASE SCHEMA

### 6.1 USERS Table

- user_id (INT, PRIMARY KEY, AUTO_INCREMENT)
- name (VARCHAR(100))
- email (VARCHAR(100))

### 6.2 CATEGORIES Table

- category_id (INT, PRIMARY KEY, AUTO_INCREMENT)
- category_name (VARCHAR(50))

### 6.3 INCOME Table

- income_id (INT, PRIMARY KEY, AUTO_INCREMENT)
- user_id (INT, FOREIGN KEY)
- amount (DECIMAL(10,2))
- date (DATE)
- source (VARCHAR(100))

### 6.4 EXPENSES Table

- expense_id (INT, PRIMARY KEY, AUTO_INCREMENT)
- user_id (INT, FOREIGN KEY)
- category_id (INT, FOREIGN KEY)
- amount (DECIMAL(10,2))
- date (DATE)
- description (VARCHAR(255))

# 7. IMPLEMENTATION

The project is implemented using MySQL. Tables are created using SQL commands, and relationships are established using foreign keys. Sample data is inserted into the tables to test the working of the system.

The database allows users to store and retrieve financial data efficiently.

# 8. SQL QUERIES

### 8.1 Display All Expenses

```
SELECT * FROM expenses;
```

### 8.2 Display All Income

```
SELECT * FROM income;
```

### 8.3 Total Expense per User

```
SELECT user_id, SUM(amount) AS total_expense
FROM expenses
GROUP BY user_id;
```

### 8.4 Category-wise Expense

```
SELECT c.category_name, SUM(e.amount) AS total_spent
FROM expenses e
JOIN categories c ON e.category_id = c.category_id
GROUP BY c.category_name;
```

### 8.4 User Expense Details

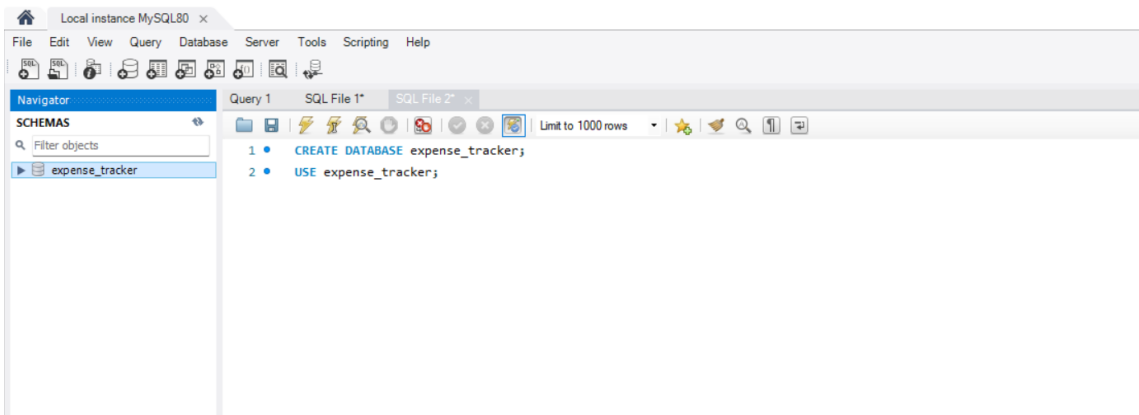
```
SELECT u.name, c.category_name, e.amount
FROM expenses e
JOIN users u ON e.user_id = u.user_id
JOIN categories c ON e.category_id = c.category_id;
```

### 8.4 Remaining Balance

```
SELECT
    u.name,
    IFNULL(SUM(i.amount), 0) - IFNULL(SUM(e.amount), 0) AS balance
FROM users u
LEFT JOIN income i ON u.user_id = i.user_id
LEFT JOIN expenses e ON u.user_id = e.user_id
GROUP BY u.name;
```

## 9. OUTPUT SCREENSHOTS

### 9.1 Database creation



## 9.2 Tables creation

Query 1 | SQL File 1* | SQL File 2*

```
SHOW TABLES;
```

Result Grid

| Tables_in_expense_tracker |
|---|
| categories |
| expenses |
| income |
| users |

## 9.3 Users Data

```
SELECT * FROM users;
```

Result Grid

| user_id | name | email |
|---|---|---|
| 1 | Ashish | ashish@gmail.com |
| 2 | Rohan | rohan@gmail.com |
| 3 | Priya | priya@gmail.com |
| NULL | NULL | NULL |

## 9.4 Categories Data

```
SELECT * FROM categories;
```

Result Grid

| category_id | category_name |
|---|---|
| 1 | Food |
| 2 | Travel |
| 3 | Shopping |
| 4 | Bills |
| 5 | Health |
| NULL | NULL |

## 9.5 Income Data

```
SELECT * FROM income;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| income_id | user_id | amount | date | source |
|---|---|---|---|---|
| 1 | 1 | 25000.00 | 2026-04-01 | Salary |
| 2 | 2 | 18000.00 | 2026-04-02 | Freelancing |
| 3 | 3 | 22000.00 | 2026-04-03 | Part-time Job |
| NULL | NULL | NULL | NULL | NULL |

## 9.6 Expenses Data

```
SELECT * FROM expenses;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

| expense_id | user_id | category_id | amount | date | description |
|---|---|---|---|---|---|
| 1 | 1 | 1 | 250.00 | 2026-04-05 | Lunch |
| 2 | 1 | 2 | 500.00 | 2026-04-06 | Auto fare |
| 3 | 1 | 4 | 1200.00 | 2026-04-07 | Electricity bill |
| 4 | 2 | 3 | 1500.00 | 2026-04-05 | Clothes shopping |
| 5 | 2 | 1 | 300.00 | 2026-04-06 | Dinner |
| 6 | 3 | 5 | 800.00 | 2026-04-08 | Medicines |
| 7 | 3 | 2 | 400.00 | 2026-04-09 | Bus ticket |
| NULL | NULL | NULL | NULL | NULL | NULL |

## 9.7 Category-wise Expense

```sql
SELECT c.category_name, SUM(e.amount) AS total_spent
FROM expenses e
JOIN categories c ON e.category_id = c.category_id
GROUP BY c.category_name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| category_name | total_spent |
|---|---|
| Food | 550.00 |
| Travel | 900.00 |
| Shopping | 1500.00 |
| Bills | 1200.00 |
| Health | 800.00 |

## 9.8 User-wise Details

```sql
SELECT u.name, c.category_name, e.amount, e.date, e.description
FROM expenses e
JOIN users u ON e.user_id = u.user_id
JOIN categories c ON e.category_id = c.category_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| name | category_name | amount | date | description |
|---|---|---|---|---|
| Ashish | Food | 250.00 | 2026-04-05 | Lunch |
| Ashish | Travel | 500.00 | 2026-04-06 | Auto fare |
| Ashish | Bills | 1200.00 | 2026-04-07 | Electricity bill |
| Rohan | Shopping | 1500.00 | 2026-04-05 | Clothes shopping |
| Rohan | Food | 300.00 | 2026-04-06 | Dinner |
| Priya | Health | 800.00 | 2026-04-08 | Medicines |
| Priya | Travel | 400.00 | 2026-04-09 | Bus ticket |

### 8.4 User Expense Details

```sql
SELECT
    u.name,
    IFNULL(i.total_income, 0) AS total_income,
    IFNULL(e.total_expense, 0) AS total_expense,
    IFNULL(i.total_income, 0) - IFNULL(e.total_expense, 0) AS balance
FROM users u
LEFT JOIN (
    SELECT user_id, SUM(amount) AS total_income
    FROM income
    GROUP BY user_id
) i ON u.user_id = i.user_id
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

| name | total_income | total_expense | balance |
|---|---|---|---|
| Ashish | 25000.00 | 1950.00 | 23050.00 |
| Rohan | 18000.00 | 1800.00 | 16200.00 |
| Priya | 22000.00 | 1200.00 | 20800.00 |

# 10. ADVANTAGES

- Easy to track expenses
- Organized data storage
- Helps in financial analysis
- Reduces manual errors
- Simple and efficient system

# 11. FUTURE SCOPE

The Personal Expense Tracker developed in this project can be further enhanced in several ways to make it more practical and user-friendly. One of the major improvements would be the addition of a graphical user interface, such as a web or mobile application, which would allow users to interact with the system more easily. Currently, the system operates using SQL queries, which may not be convenient for all users.

Additionally, authentication and user login functionality can be implemented to ensure data privacy and security. Each user would be able to access and manage only their

own financial data. The system can also be extended to include graphical representations such as charts and dashboards for better visualization of income and expenses.

Another potential enhancement is integration with banking APIs or financial services to automatically fetch transaction data, reducing manual data entry. Features like budget setting, expense alerts, and predictive analysis using machine learning techniques can also be added to make the system more intelligent and efficient. Overall, these improvements would transform the project into a complete financial management system.

# 12. CONCLUSION

The Personal Expense Tracker project successfully demonstrates the practical implementation of Database Management System concepts. Through this project, a structured database was designed to store and manage financial data efficiently. The use of tables, relationships, and SQL queries allowed for effective organization and analysis of income and expenses.

The system enables users to track their financial activities, categorize expenses, and generate useful insights such as total spending and remaining balance. This helps in better financial planning and decision-making. The project also highlights the importance of data organization and retrieval in real-world applications.

In conclusion, this project not only fulfills its objective of managing personal financial data but also provides a strong foundation for developing more advanced financial systems in the future.